

Ex. No: 5 Inter Process Communication (IPC)

C PROGRAM:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <string.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    int fd[2];
```

```
    pid_t pid;
```

```
    char message[] = "Hello from Child Process";
```

```
    char buffer[100];
```

```
    if (pipe(fd) == -1)
```

```
    {
```

```
        perror("Pipe failed");
```

```
        return 1;
```

```
    }
```

```
    pid = fork();
```

```
    if (pid < 0)
```

```
    {
```

```
        perror("Fork failed");
```

```
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        close(fd[0]);

        write(fd[1], message, strlen(message) + 1);

        close(fd[1]);
        exit(0);
    }
    else
    {
        // Parent process
        wait(NULL);

        close(fd[1]);

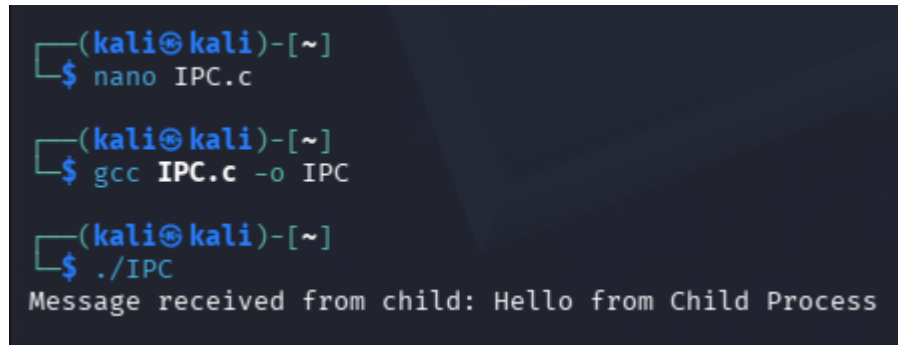
        read(fd[0], buffer, sizeof(buffer));

        printf("Message received from child: %s\n", buffer);

        close(fd[0]);
    }
```

```
    return 0;
}
```

Output:

A terminal window with a dark background and light blue text. The prompt is (kali㉿kali)-[~]. The user enters 'nano IPC.c', then 'gcc IPC.c -o IPC', and finally './IPC'. The output of the last command is 'Message received from child: Hello from Child Process'.

```
(kali㉿kali)-[~]
$ nano IPC.c

(kali㉿kali)-[~]
$ gcc IPC.c -o IPC

(kali㉿kali)-[~]
$ ./IPC
Message received from child: Hello from Child Process
```

SHELL SCRIPT:

```
#!/bin/bash
```

```
echo "Hello from Child Process" | cat
```

Alternative Shell Script

```
#!/bin/bash
```

```
(
```

```
echo "Message from Child Process"
```

```
) | while read msg
```

```
do
```

```
echo "Parent Received: $msg"
```

```
done
```

Output:

```
(kali㉿kali)-[~]  
$ nano lab-5.sh
```

```
(kali㉿kali)-[~]  
$ chmod +x lab-5.sh
```

```
(kali㉿kali)-[~]  
$ bash lab-5.sh  
Parent Received: Message from Child Process
```